Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Sorting with Python tools

Today
Welcome to your next lesson: Sorting with Python tools.

I'm getting your lesson ready — based on what you've learned, what's coming up, and the best way for you to practice.

Step by step, you’re moving forward.

Imagine you have a list of game scores in Python:

Python

```
scores = [50, 10, 75, 30]
```

You want them from lowest to highest. You could write another bubble or insertion sort… or you can let Python do the work in one step.

In Python there are two main ways to sort a list:

```
sorted(scores)
scores.sort()
```

We’ll dig into both, plus key= and reverse=, and when to trust Python’s fast sort vs. writing your own.

To start simple: if you run this in the editor:

Python

```
scores = [50, 10, 75, 30]
result = sorted(scores)
print("scores:", scores)
print("result:", result)
```

What’s your guess about the output — will scores change, or will only result be sorted?

result will change, score will not as it not a inline likelist.sort , it returns a value

Exactly right 🎯

sorted(scores) → returns a new sorted list, does not change scores.
scores.sort() → sorts the same list in place (mutates it) and returns None.

So for:

Python

```
scores = [50, 10, 75, 30]
result = sorted(scores)
print("scores:", scores)
print("result:", result)
```

You'd see:

```
scores: [50, 10, 75, 30]
result: [10, 30, 50, 75]
```

Next step: I want you to try this yourself in the code editor.

In the editor, write code that:

Creates a list nums = [3, 1, 4, 1, 5]
Uses nums.sort()
Prints nums
Run it, then submit the code.

After you've done that: what do you notice about nums after .sort()?

it itself is sorted and the return value if assigned is None

Perfect 👌 You've got the key difference:

sorted(nums) → returns a new sorted list, original stays the same.
nums.sort() → changes nums itself and returns None.
Next concept: sorting in reverse order 🔁

With sorted, you can do:

Python

```
nums = [3, 1, 4, 1, 5]
result = sorted(nums, reverse=True)
```

This gives you a new list from largest to smallest.

Python's built-in sort is ascending by default, and you flip it with reverse=True.

Does that idea make sense: "ascending by default, reverse=True for descending"?

If yes, I'll next show you how key= works to sort by things like string length or object fields.

Yes, that makes sense

Nice 👍 let's build on that with one new idea: key=.

Sometimes you don't want to sort by the value itself, but by something about the value.

Example: sorting words by their length instead of alphabetically.

Step by step:

Python

```
words = ["apple", "kiwi", "banana", "fig"]
result = sorted(words, key=len)
print(result)
```

Here, key=len tells Python:

“Before comparing, turn each item into its length, and sort by that.”

So it sorts by: [5, 4, 6, 3] → and reorders the words to match.

Check-in:

Does this mental picture work for you?

“Sort by the result of key(some_item)”
In this case, key=len means “sort by length”

If that feels okay, I’ll have you try a small key= exercise next.

Yes, that picture helps

Great, then let’s use that picture with a concrete list 🔍

We’ll sort a list of small dictionaries (like tiny records):

Python

```
players = [
    {"name": "Alice", "score": 30},
    {"name": "Bob", "score": 50},
    {"name": "Cara", "score": 40},
]
```

If I write:

Python

```
sorted_players = sorted(players, key=lambda p: p["score"], reverse=True)
```

step by step this means:

For each p in players, look at p["score"].
Sort by that number.
reverse=True → highest score first.

So we’re saying: “sort players by score, descending.”

Before you run anything, just think it through:

In that sorted order, which name do you expect to come first, then second, then third?

cara then bob then alice, reversed

Nice catch to think about it carefully 👀

Let’s line up the scores:

Alice → 30
Bob → 50
Cara → 40

We sort by score, descending (reverse=True), so from largest to smallest:

Bob (50)

Cara (40)
Alice (30)

So the correct order is: Bob, Cara, Alice.

Your “Cara then Bob then Alice, reversed” sounds like you were picturing a reverse of something, but the actual reverse=True here already means “highest score first.”

Want to try a tiny coding task with this idea?

In the code editor, write code that:

Creates the players list above.
Uses sorted(players, key=lambda p: p["name"]) to sort alphabetically by name.
Prints the sorted result.

Run it, then submit your code.

After that, tell me: which name comes first in that output?

Python

```
players = [
    {"name": "Alice", "score": 30},
    {"name": "Bob", "score": 50},
    {"name": "Cara", "score": 40},
]

sorted_players = sorted(players, key=lambda p: p["score"], reverse=True)
print(sorted_players)
```

Nice work running that 🔥

You sorted by score descending, so the result:

Python

```
[{'name': 'Bob', 'score': 50},
 {'name': 'Cara', 'score': 40},
 {'name': 'Alice', 'score': 30}]
```

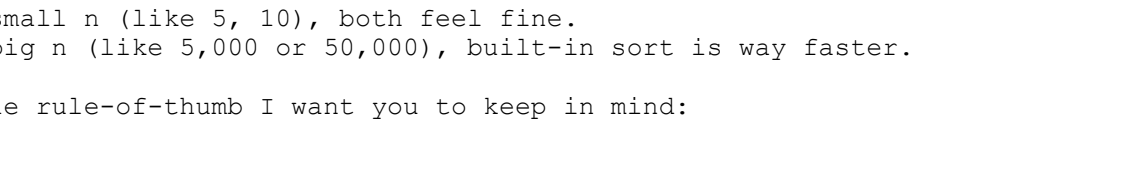
matches exactly what we predicted: Bob, Cara, Alice.

Now let’s zoom out to the “why use built-in sort?” idea.

You already know bubble/insertion sort are about $O(n^2)$ — they get slow as lists grow.

Python’s built-in sorted / .sort() use a much faster algorithm (called Timsort) that runs in about O(n log n).

Visually:



For small n (like 5, 10), both feel fine.
For big n (like 5,000 or 50,000), built-in sort is way faster.

Simple rule-of-thumb I want you to keep in mind:

In real code, almost always use the built-in sort, not your own bubble/insertion, especially once lists are more than ~30 items or when you care about clean, readable code.

Does that rule make sense to you?

If yes, I'll give you a quick "choose the best tool" question to lock it in.

Yes, that makes sense

What do you think is the best option here? 👇

You're writing Python code that needs a list of 10,000 user scores sorted from highest to lowest, in a clear and fast way. What's the best choice?

A.

Write your own bubble sort so you control every step

B.

Use sorted(scores, reverse=True) to let Python handle it

C.

Use a slow manual loop that repeatedly finds the max

D.

Avoid sorting at all because 10,000 is too many items

Nice choice, Ron 🎯

Using sorted(scores, reverse=True) is exactly what a strong Python dev would do here:

It's fast: under the hood it uses Python's optimized O(n log n) sort, way faster than bubble or a "find max in a loop" approach for 10,000 items.
It's clear: one short line shows intent — "sort this list descending" — which makes your code easy to read and maintain.
Before we wrap this lesson, I want to quickly check one last detail with you:

If you already have a list scores, how would you sort it in place from lowest to highest using the list method (not sorted)?

You can just answer in words, like: "Call … on …"

cll scores.sort()

Exactly ☑

To sort in place, ascending: scores.sort()
To get a new sorted list and keep scores unchanged: sorted(scores)

You've now hit all the key ideas for this lesson:

sorted() vs .sort() (copy vs in-place)
reverse=True for descending
key= (including lambda) to control how things are sorted
Built-in sort is O(n log n) and is usually the right tool instead of hand-written $O(n^2)$ sorts
You've basically mastered this topic 🎓

Are you ready to wrap up this lesson and move on to the next one, or do you want to try one more practice challenge with key and reverse first?

Wrap up and move on

Nice work, Ron 🎉

Quick recap of what you're taking with you from this lesson:

You can choose between sorted() (returns a new list) and .sort() (changes the list in place).
You know how to control order with reverse=True for descending.
You can customize sorting with key=..., including lambda functions (like sorting players by score or name).
You understand that Python's built-in sort is about O(n log n) and is the right tool for real-world code, instead of manual O(n^2) bubble/insertion sorts.
You're building real "developer instincts" now — choosing the right tool instead of just knowing the theory 💪

When you're ready, go ahead and jump into the next lesson in the Learn tab. If anything from this one feels fuzzy later, come back and I'll walk through it with you again step by step.

```
nums = [3, 1, 4, 1, 5]
sorted_num = nums.sort()
print(nums)
print(sorted_num)


players = [
    {"name": "Alice", "score": 30},
    {"name": "Bob", "score": 50},
    {"name": "Cara", "score": 40},
]

sorted_players = sorted(players, key=lambda p: p["score"], reverse=True)
print(sorted_players)
```